

UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA



Programación II - Laboratorio

DEFENSA DEL JARDÍN

Implementación del Juego en C++

Informe de Proyecto

INTEGRANTES	SECCIÓN	PARTICIPACIÓN
Arnie Farid Espinoza Pachas	Teo 1 - Lab 12	100%
Milagros Brisset Cruzado Saenz	Teo 1 - Lab 12	100 %
Gabriel Junior Tadeo Reyes	Teo 1 - Lab 12	100%

Lima - Perú

2026-1

Antecedentes

1.1 Contexto y Motivación

Los videojuegos de estrategia en tiempo real y por turnos han sido una de las categorías más influyentes en la historia del entretenimiento digital. Dentro de este género, los juegos de tipo "Tower Defense" han ganado especial popularidad por combinar pensamiento estratégico, gestión de recursos y acción progresiva. El referente más icónico de este subgénero es Plants vs. Zombies (PopCap Games, 2009), cuya mecánica de defensa por carriles inspiró el presente proyecto.

En el contexto académico del curso Programación II de la Universidad de Ingeniería y Tecnología (UTEC), este proyecto representa una oportunidad concreta de aplicar los conceptos más avanzados de la Programación Orientada a Objetos (POO) en C++ sobre un problema de naturaleza lúdica e interactiva. El reto no radica únicamente en hacer funcionar el juego, sino en diseñarlo con una arquitectura limpia, extensible y orientada a objetos, donde cada decisión de diseño responda a un principio técnico justificable.

1.2 Planteamiento del Problema

El jardín de la Academia está siendo invadido por hordas de zombis que avanzan fila por fila hacia la casa del jugador. La defensa requiere colocar plantas estratégicamente en una cuadrícula ASCII, gestionando un recurso llamado "soles" y enfrentando oleadas de enemigos con comportamientos diferenciados. El sistema debe ser completamente funcional en consola, sin librerías gráficas, y persiste los resultados de cada partida en disco para consultas posteriores.

1.3 Alcance del Proyecto

- Juego de consola en C++ con interfaz ASCII completamente interactiva.
- 3 tipos de plantas con comportamientos diferenciados (Girasol, Lanzaguisantes, Cactus).
- 3 tipos de zombis con habilidades pasivas propias (Básico, Casco, Rápido).
- Sistema de oleadas progresivas (3 oleadas por defecto, escalable).

- Persistencia de estadísticas en archivo de texto (historial_partidas.txt).
- Aplicación de herencia, polimorfismo, sobrecarga de operadores, composición y agregación.

2. Fundamento Teórico

2.1 Programación Orientada a Objetos (POO)

La Programación Orientada a Objetos es un paradigma de programación que organiza el software en torno a objetos, los cuales encapsulan datos (atributos) y comportamiento (métodos). Sus cuatro pilares fundamentales son:

- Encapsulación: los atributos internos de una clase se protegen mediante modificadores de acceso (`private`, `protected`) y se accede a ellos a través de métodos públicos (`getters/setters`). En el proyecto, los atributos como `hp`, `danio` y `posicionColumna` son privados o protegidos en todas las clases.
- Herencia: permite definir clases derivadas que heredan atributos y métodos de una clase base, reutilizando código. En el proyecto, `Girasol`, `Lanzaguisantes` y `Cactus` heredan de `Planta`; `ZombieBasico`, `ZombieCasco` y `ZombieRapido` heredan de `Zombie`.
- Polimorfismo: la capacidad de que objetos de distintas clases derivadas respondan de forma diferente a la misma llamada de método. Se implementa mediante métodos virtuales (`virtual`) y punteros/referencias a la clase base.
- Abstracción: se modela solo lo esencial del dominio del problema. Las clases `Planta` y `Zombie` definen contratos (interfaces) mediante métodos virtuales puros que cada subclase debe implementar.

2.2 Herencia y Polimorfismo en C++

En C++, la herencia se declara con la sintaxis `class Derivada : public Base`. El polimorfismo dinámico se habilita declarando métodos como `virtual` en la clase base, lo que permite que el compilador resuelva la llamada correcta en tiempo de ejecución mediante la tabla de métodos virtuales (`vtable`). El destructor virtual (`~Planta()`) =

`default`) es fundamental para garantizar la destrucción correcta de objetos polimórficos.

El método virtual puro (`virtual void actuar(...) = 0`) convierte a `Planta` en una clase abstracta, obligando a cada subclase a proveer su propia implementación de `actuar()`, que define el comportamiento particular de cada planta en cada turno.

2.3 Sobrecarga de Operadores

C++ permite redefinir el comportamiento de operadores estándar para tipos definidos por el usuario. Esto mejora la legibilidad del código al permitir expresiones naturales como `*planta -= 20` (aplicar daño) o `cout << partida` (imprimir información). La sobrecarga puede implementarse como método miembro de la clase o como función amiga (`friend`).

2.4 Relaciones entre Clases

- Composición: relación donde el objeto contenido no puede existir sin el contenedor. El `Tablero` posee y gestiona la cuadrícula de plantas; si el `Tablero` se destruye, las plantas también.
- Agregación: el objeto contenido puede existir independientemente. Los zombis se asocian temporalmente a filas del `Tablero` y pueden ser eliminados sin destruir el `Tablero`.
- Asociación: relación de uso entre clases sin implicar propiedad. La clase `Juego` utiliza `Tablero` pero podría existir sin él.

2.5 Gestión de Memoria con Punteros Inteligentes

C++11 introdujo los punteros inteligentes (`std::shared_ptr`, `std::weak_ptr`) que automatizan la liberación de memoria dinámica, eliminando la necesidad de `delete` manual y previniendo `memory leaks`. En el proyecto se usa `std::shared_ptr<Planta>` y `std::shared_ptr<Zombie>` para gestionar el ciclo de vida de las entidades del juego de forma segura.

2.6 Juegos de Torre de Defensa (Tower Defense)

Un Tower Defense es un subgénero de juegos de estrategia donde el objetivo es defender un punto de llegada colocando estructuras defensivas a lo largo de un camino por el que avanzan oleadas de enemigos. Las mecánicas fundamentales incluyen: gestión de recursos para construir/mejorar defensas, oleadas progresivas de enemigos con mayor dificultad, y balance entre distintos tipos de defensas y amenazas. En versiones por turnos, como la de este proyecto, el jugador tiene tiempo para planificar antes de que los enemigos actúen.

3. Métodos y Desarrollo

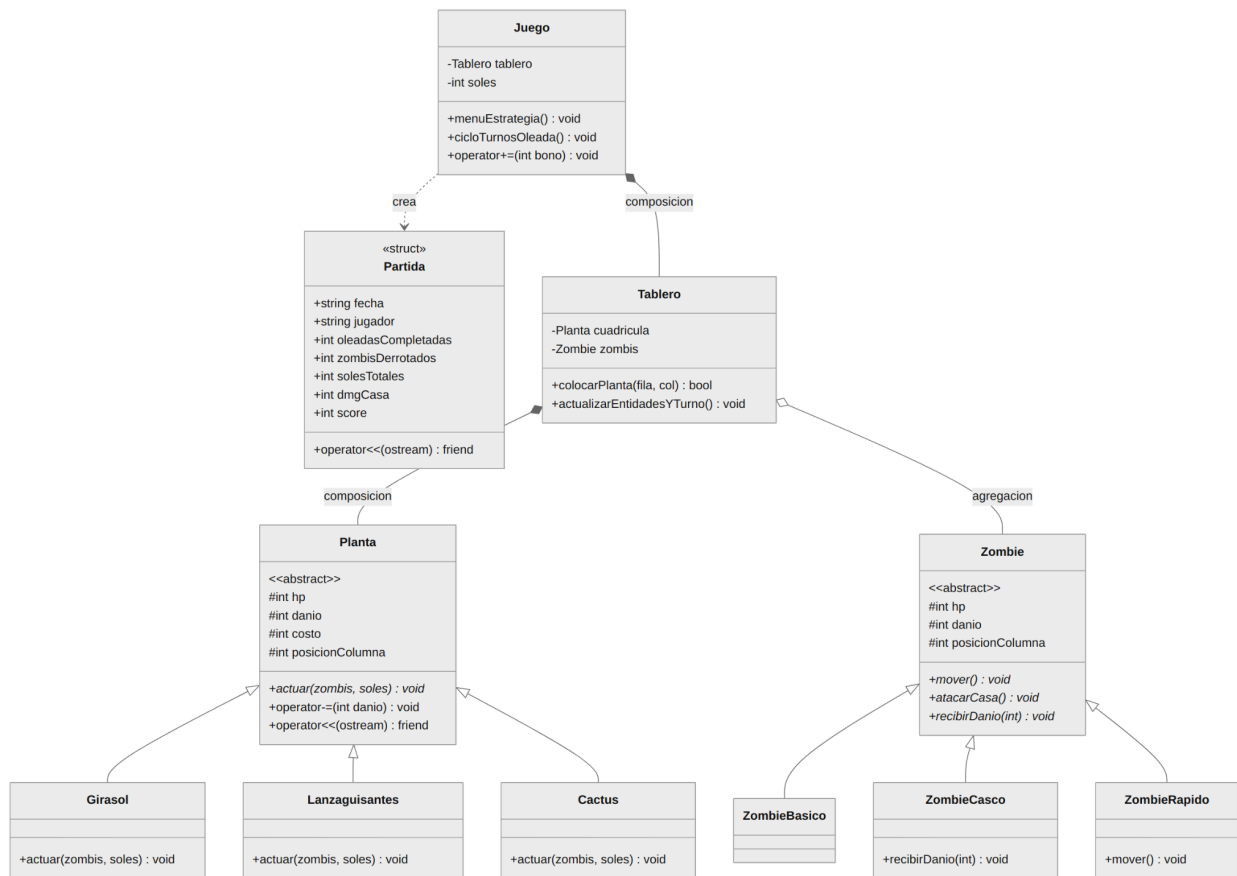
3.1 Arquitectura del Sistema

El proyecto se estructura en cinco módulos independientes siguiendo el principio de responsabilidad única. Cada módulo se separa en un *archivo de cabecera (.h)* con las declaraciones y un archivo de implementación (*.cpp*) con las definiciones:

Módulo / Archivo	Clases	Responsabilidad
main.cpp	[-]	Punto de entrada. Solo crea el objeto Juego y llama al bucle principal.
Juego.h / Juego.cpp	Juego, Partida	Controlador principal. Gestiona el menú, oleadas, ciclo de turnos y guardado de estadísticas.
Tablero.h / Tablero.cpp	Tablero	Cuadrícula del jardín. Maneja la colocación de plantas, movimiento de zombis y procesamiento de turnos.
Planta.h / Planta.cpp	Planta, Girasol, Lanzaguisantes , Cactus	Jerarquía de plantas con comportamiento polimórfico por turno.
Zombie.h / Zombie.cpp	Zombie, ZombieBasico, ZombieCasco, ZombieRapido	Jerarquía de zombis con movimiento y recepción de daño polimórficos.

3.2 Diagrama de Clases

A continuación se describe la jerarquía y relaciones entre las clases del sistema:



3.3 Jerarquía de Plantas

La clase base **Planta** define la interfaz común de todas las plantas mediante el método virtual puro `actuar()`, que recibe la lista de zombis presentes en la fila y la cantidad actual de soles del jugador:

Clase	HP	Costo (soles)	Daño	Comportamiento (<code>actuar()</code>)
Girasol	60	50	0	Habilidad pasiva: genera 25 soles cada 2 turnos automáticamente.
Lanzaguisantes	100	100	20	Dispara al primer zombi de la fila cada turno, infligiendo 20 de daño.

Cactus	150	125	15	Lanza espinas al primer zombi de la fila cada turno. Mayor HP defensivo.
--------	-----	-----	----	--

3.4 Jerarquía de Zombis

La clase base `Zombie` define los métodos virtuales `mover()`, `atacarCasa()` y `recibirDanio()`, los cuales son sobrescritos por las subclases para implementar comportamientos diferenciados:

Clase	HP	Daño	Velocidad	Habilidad Especial
ZombieBasico	100	15	1 col/turno	Sin habilidad. Enemigo estándar de referencia.
ZombieCasco	120	15	1 col/turno	Armadura pasiva: reduce el daño recibido al 50% durante los primeros 3 ataques (sobreescribe <code>recibirDanio()</code>).
ZombieRapido	80	10	1 col/turno + bonus	Agilidad pasiva: cada 3 turnos avanza una columna adicional (sobreescribe <code>mover()</code>).

3.5 Lógica del Juego y Ciclo de Turnos

El flujo de ejecución del juego respeta el orden de turno especificado en el enunciado. La clase `Tablero`, a través del método `actualizarEntidadesYTurno()`, implementa la siguiente secuencia en cada turno:

1. Fase de Plantas: cada planta en la cuadrícula ejecuta su método `actuar()` (genera soles o ataca al primer zombi en su fila).
2. Limpieza intermedia: los zombis con `HP <= 0` son eliminados de la lista y se contabilizan como derrotados.
3. Fase de Zombis: cada zombi activo verifica su posición. Si está en la columna 0, ataca la casa; si hay una planta en la columna inmediatamente anterior, la ataca; en caso contrario, ejecuta su `mover()`.

4. Comprobaciones de estado: se evalúa si quedaron zombis activos y si la casa sigue en pie (`hpCasa > 0`).

El flujo general de la partida es el siguiente:

- El jugador ingresa su alias y el juego inicia la Oleada 1.
- Fase de preparación (`menuEstrategia()`): el jugador coloca o elimina plantas usando los soles disponibles.
- El sistema spawnea zombis aleatorios en filas aleatorias desde la columna más lejana (columna 8).
- Ciclo de turnos (`cicloTurnosOleada()`): se repite hasta que no queden zombis activos o la casa caiga.
- Al superar una oleada, el jugador recibe 75 soles de bonificación y pasa a la siguiente.
- Al terminar la partida (victoria o derrota), se guardan las estadísticas en `historial_partidas.txt`.

3.6 Sobrecarga de Operadores

El proyecto implementa tres sobrecargas de operadores que cumplen el requisito técnico del enunciado:

Operador	Clase / Contexto	Descripción y Uso
<code>operator<<</code>	Planta, Zombie, Partida	Permite imprimir el estado de cualquier entidad directamente con <code>cout</code> . Para plantas y zombis muestra inicial + HP en formato ASCII (ej: G(60)). Para Partida serializa todos los campos al historial de texto.
<code>operator-=</code>	Planta, Zombie	Aplica daño reduciendo el HP de la entidad. Garantiza que el HP no baje de 0. Permite expresiones como

		<code>*planta -= 20</code> o <code>*zombie -= danio</code> , haciendo el código de combate más legible.
<code>operator+=</code>	Juego	Añade soles directamente al contador del jugador y al total recolectado. Usado para la bonificación entre oleadas: <code>*this += 75</code> .

3.7 Visualización del Mapa (ASCII)

El tablero se renderiza en consola con una cuadrícula de 5 filas x 9 columnas. Cada celda muestra la inicial del tipo de entidad seguida de su HP entre paréntesis. Las celdas vacías se muestran como [.]. El mapa se actualiza en cada turno, permitiendo al jugador visualizar la posición de plantas y zombies en tiempo real.

Ejemplo de visualización durante una partida:

```

===== MAPA DEL JARDÍN (ASCII) =====
Fila 0 | G(60)  [ . ]  [ . ]  [ . ]  [ . ]  [ . ]  [ . ]  [ . ]
Z(85) |
Fila 1 | L(100) [ . ]  [ . ]  [ . ]  [ . ]  [ . ]  [ . ]  C(95)  [
. ] |
Fila 2 | [ . ]  [ . ]  [ . ]  [ . ]  [ . ]  [ . ]  [ . ]  [ . ]  [
. ] |
Fila 3 | C(150) [ . ]  [ . ]  [ . ]  [ . ]  [ . ]  [ . ]  [ . ]
Z(120) |
Fila 4 | [ . ]  [ . ]  [ . ]  [ . ]  [ . ]  [ . ]  [ . ]  [ . ]  [
. ] |
=====

```

3.8 Persistencia de Estadísticas

Al finalizar cada partida, el sistema guarda un registro en el archivo `historial_partidas.txt` usando la sobrecarga del operador `<<` de la estructura `Partida`. Cada línea almacena: fecha, jugador, oleadas completadas, zombies eliminados, soles recolectados, daño

recibido por la casa y puntuación final. El menú "Ver estadísticas" lee este archivo y lo muestra numerado en consola.

Ejemplo de línea guardada en el historial:

```
Fecha: 2026-06-16 17:30 | Jugador: Alumno1 | Oleadas: 3/3 |  
Derrotados: 12 | Soles Totales: 450 | Dmg Casa: 20 | Score:  
3300
```

3.9 Pruebas Realizadas

Se realizaron las siguientes pruebas funcionales para validar el correcto comportamiento del sistema:

#	Caso de Prueba	Entrada / Condición	Resultado Esperado y Obtenido
1	Colocación de planta con soles insuficientes	Soles = 30, intentar colocar Lanzaguisantes (costo 100)	Mensaje de error; no se coloca la planta.
2	Colocación en columna de spawn (columna 8)	Coordenadas (2, 8)	La celda se rechaza; <code>Tablero::colocarPlanta</code> retorna <code>false</code> .
3	Generación de soles por Girasol	Girasol colocada, esperar 2 turnos	Se añaden 25 soles al jugador en el turno 2.
4	Armadura de <code>ZombieCasco</code>	Lanzaguisantes (20 dmg) ataca a <code>ZombieCasco</code>	Daño reducido a 10 los primeros 3 ataques. HP: 120 → 110 → 100 → 90.
5	Movimiento doble de <code>ZombieRapido</code>	<code>ZombieRapido</code> activo, turno múltiplo de 3	El zombi avanza 2 columnas en ese turno por pasiva de agilidad.
6	Derrota por HP de <code>casa = 0</code>	Zombi llega a columna 0 y ataca la casa hasta <code>HP = 0</code>	El juego muestra el mensaje de derrota y guarda estadísticas.

7	Victoria al completar 3 oleadas	Superar oleadas 1, 2 y 3 sin que la casa caiga	Mensaje de victoria y guardado de estadísticas.
8	Entrada inválida en menú	Ingresar texto ("abc") en lugar de número	<code>cin.clear() + cin.ignore()</code> evitan bloqueo; se muestra mensaje de error.
9	Historial vacío	Ver estadísticas sin partidas previas	Mensaje informativo: "No existen partidas registradas".
10	Bonificación entre oleadas con <code>operator+=</code>	Superar oleada 1	El jugador recibe 75 soles adicionales mediante <code>*this += 75.</code>

4. Conclusiones y Trabajo Futuro

4.1 Conclusiones

El desarrollo del proyecto "Defensa del Jardín" permitió aplicar de forma integral y práctica los conceptos avanzados de la Programación Orientada a Objetos en C++. A continuación, las principales conclusiones:

- La herencia y el polimorfismo resultaron ser las herramientas más poderosas del diseño: definir `actuar()` como virtual puro en `Planta`, y `mover()` / `recibirDanio()` como virtual en `Zombie`, permitió que el Tablero procesara todas las entidades sin conocer su tipo concreto, cumpliendo el principio de sustitución de Liskov (Ferrer, B. (s.f.)).
- La separación en módulos (un `.h` y un `.cpp` por entidad) facilitó el desarrollo paralelo y redujo las dependencias circulares. La declaración adelantada (`class Zombie;`) en `Planta.h` fue clave para romper la dependencia mutua entre `Planta` y `Zombie`.
- El uso de `std::shared_ptr<Planta>` y `std::shared_ptr<Zombie>` eliminó la necesidad de gestión manual de memoria (`delete`), previniendo

memory leaks de forma automática y haciendo el código más seguro y mantenible.

- La sobrecarga de operadores (`-=`, `+=`, `<<`) mejoró significativamente la legibilidad del código de combate y serialización. Expresiones como `*enemigo -= danio` comunican la intención de forma más clara que llamar a un método como `enemigo->recibirDanio(danio)`.
- La clase `Tablero` actúa efectivamente como mediador entre plantas y zombies, encapsulando toda la lógica de colisión y procesamiento de turno, lo que mantiene las clases `Planta` y `Zombie` libres de conocerse mutuamente (excepto por la declaración adelantada necesaria para `actuar()`).
- La validación de entradas en todos los puntos de interacción con el usuario demostró ser esencial para la robustez del sistema, evitando que entradas inválidas bloqueen el programa mediante el patrón `cin.clear() + cin.ignore()`.

4.2 Trabajo Futuro

Como extensiones naturales del proyecto que enriquecerían la experiencia sin comprometer la arquitectura actual, se propone:

- Ampliar la variedad: agregar al menos 2 plantas y 2 zombies adicionales con habilidades activas (explosión en área, congelación total, salto de casilla) aprovechando la jerarquía polimórfica ya establecida.
- Sistema de mejora de plantas: implementar niveles para cada planta (nivel 1, 2, 3) con estadísticas incrementales, usando sobrecarga del operador `++` para subir de nivel.
- Configuración de dificultad: permitir al jugador elegir entre fácil, normal y difícil al inicio de la partida, ajustando la cantidad y tipo de zombies por oleada.
- Guardado y carga de partidas: persistir el estado completo de una partida en progreso (posición de plantas, HP de casa, oleada actual) para poder reanudarla en sesiones futuras.

- Modo de dos jugadores: un jugador controla las plantas y otro controla los zombis en el mismo turno, aprovechando la arquitectura por capas del sistema.
- Interfaz mejorada con colores ANSI: usar códigos de escape ANSI para colorear el mapa en consola, diferenciando plantas (verde), zombis (rojo) y la casa (amarillo) sin necesidad de librerías gráficas externas.

Referencias

cppreference.com. (2024). C++ reference. <https://cppreference.com>

PopCap Games. (2009). Plants vs. Zombies [Videojuego]. Electronic Arts.

Ferrer, B. (s.f.). *Principios SOLID: (3) Liskov Substitution Principle*. Secture. <https://secture.com/principios-solid-3-liskov-substitution-principle/>